

Product overview

Summary

ReadySetFly is a dual marketplace for general aviation:

- Rentals marketplace (Airbnb-style) for aircraft hourly bookings.
- Services marketplace (classifieds-style) for sales, CFIs, schools, mechanics, charter, and jobs.

Core value proposition

- Single account for pilots, owners, and service providers.
- Trust layer with identity and document verification.
- Messaging gate tied to rental status to reduce fraud and privacy risk.

Pilot tools and training suite

ReadySetFly takes a pilot-tools-first approach alongside the marketplaces, including:

- Digital logbook with endorsements and signatures.
- Flight planner with weather overlays.
- Student training hub (roadmaps, written prep, progress tracking).
- VOR trainer and GPS sims.
- Radio communications trainer and checklists.
- TFR map and approach plates.

Trust and safety layers

- Email and phone verification.
- Government ID (front/back) and selfie liveness check.
- Optional FAA certificate verification.
- Payment method on file requirement.
- Admin suspension for expired documents.

Business model

- Rentals: platform fee percentage per booking.
- Marketplace listings: tiered paid listings (duration-based).

Current status (from internal analysis)

- Web app, backend, and database are production-ready.
- Key blockers remain for owner payouts, messaging E2E validation, and automated rental completion.

TBD / confirm

- Active users, listings, and revenue to date.
- Customer acquisition metrics and retention.

Marketplaces and revenue model

Rentals marketplace

- Owners list aircraft with hourly rates, requirements, and availability.
- Renters request bookings with date/hour selection and pay via PayPal.
- Messaging opens only during active rentals to reduce abuse.
- Reviews are enabled post-rental.

Services marketplace

Categories supported:

- Aircraft sales
- Charter services
- Flight instructors (CFI)
- Flight schools
- Aircraft mechanics
- Aviation jobs

Pricing and monetization

- Rentals: platform fee percentage split between renter and owner.
- Marketplace: tiered paid listings with durations and upgrades.

Payments and payouts

- PayPal Orders API for checkout.
- Payouts to owners require Braintree Marketplace approval (blocker).

TBD / confirm

- Actual fee percentages in production.
- Average listing price and conversion rates.
- Current GMV and take rate.

Tech stack and architecture

Web

- React 18 + Vite
- TypeScript
- Tailwind CSS + Radix UI
- TanStack Query
- Wouter routing
- Leaflet for mapping (no Mapbox)

Backend

- Node.js + Express
- TypeScript
- Drizzle ORM
- WebSockets for real-time messaging

Data and storage

- PostgreSQL (Neon serverless)
- Object storage for user uploads
- Schema shared between web and mobile (TypeScript)

Payments

- PayPal Orders API
- Braintree (payouts / marketplace approval pending)

AI

- OpenAI API integration for content assistance (listing descriptions)

Deployment

- Backend on Render
- Frontend on GitHub Pages (pages build)

TBD / confirm

- Current CI/CD pipeline details.
- Environment isolation (dev/stage/prod) and secrets management.

Web app routes and modules

Public routes

Key public routes from the app router:

- / (landing)
- /rentals
- /marketplace
- /aircraft/:id
- /pilot-tools
- /flight-planner
- /approach-plates
- /tfr-map
- /ownership-cost-calculator
- /weight-balance
- /radio-comms-trainer
- /adsb-receiver-help
- /gps-sims
- /student (and student sub-routes)
- /ifr-tools
- /faq
- /events
- /privacy-policy
- /terms-of-service

Protected routes (auth required)

- /dashboard
- /profile
- /messages
- /favorites
- /my-listings
- /list-aircraft
- /create-marketplace-listing
- /marketplace/listing/checkout
- /rental-payment/:id
- /owner-payout-setup
- /owner-withdrawals
- /verify-identity
- /logbook
- /notifications

- /admin (role-based)

Major feature areas

- Rentals: search, booking, payments, messaging, reviews
- Marketplace: listing creation, paid tiers, upgrades, checkout
- Pilot tools: weather, planning, training utilities
- Admin: users, listings, verifications, banners, analytics

TBD / confirm

- Admin role matrix and default permission sets.

Mobile app status

Overview

- React Native (Expo) app for iOS and Android.
- Uses the same backend API and shared TypeScript schemas.
- Targets parity with web, excluding admin.

Current capabilities

- Rentals browsing, booking, and payments
- Marketplace browsing and listing creation
- Profile management, verification status
- Balance and withdrawals
- Real-time messaging UI shell

Known parity gaps (from internal analysis)

- Favorites / wishlist
- Reviews system (create and view)
- Messaging (functional, real-time)
- Pilot tools (weather, logbook, resources)
- Ownership cost calculator
- Some profile sub-screens (my listings, personal info)

Setup note

- Requires react-native-webview install for payments.

TBD / confirm

- Current App Store / Play Store status.
- Mobile analytics and retention.

Operations and deployments

Environments

- Production backend hosted on Render.
- Frontend static hosting on GitHub Pages.

Observability and monitoring

- TBD (confirm logging, error tracking, uptime monitoring).

Data and compliance

- User identity verification and document storage.
- Document expiry enforcement via admin suspension.

Support workflows

- TBD (support inbox, ticketing, escalation).

TBD / confirm

- Backup/restore strategy.
- Incident response playbook.
- Data retention and deletion policy.

Risks, blockers, and roadmap

Critical blockers (from internal analysis)

1. Owner payouts not enabled
 - Requires Braintree Marketplace approval.
 - Owners cannot withdraw earnings until enabled.
2. Messaging end-to-end validation
 - Messaging state machine appears correct but needs full E2E test.
3. Automated rental completion
 - No cron to auto-close rentals at end date.

Medium-priority risks

- Limited rate limiting on some endpoints.
- Some file upload validation gaps.
- Potential data inconsistencies on payment failure.

Suggested near-term roadmap

- Enable Braintree Marketplace and payouts.
- Add rental auto-complete cron job.
- Run full E2E rental test (including messaging and payouts).
- Add rate limiting and upload validation hardening.
- Close mobile parity gaps (messaging, reviews, favorites).

TBD / confirm

- Security review results.
- Pen test or audit history.

Live site screenshots

Screenshots are generated from public routes and stored in /dataroom/screenshots.

Included routes

- /
- /rentals
- /marketplace
- /student
- /student/vor-trainer
- /gps-sims
- /flight-planner

Files

- /dataroom/screenshots/desktop/*.png
- /dataroom/screenshots/mobile/*.png

Generate / refresh

Run:

- Node: dataroom/scripts/capture-screenshots.mjs

Update this file if routes or filenames change.

ReadySetFly data room packet

This folder contains a concise, buyer-focused packet describing the product, stack, and current status.

Contents

- 01_product_overview.md
- 02_marketplaces_revenue.md
- 03_tech_stack_architecture.md
- 04_web_app_routes.md
- 05_mobile_app.md
- 06_operations_deployments.md
- 07_risks_blockers_roadmap.md
- 08_live_site_screenshots.md

Generated outputs

- /dataroom/out (PDF exports)
- /dataroom/screenshots (live-site images)

How to generate PDFs

- PowerShell: dataroom/scripts/print-pdfs.ps1
- Bash: dataroom/scripts/print-pdfs.sh

How to capture screenshots

- Node: dataroom/scripts/capture-screenshots.mjs

Update any TBD fields as new facts become available.